

ГЕНЕРАЦИЯ ШАБЛОНОВ ПРОВЕРОК ДЛЯ СКАНЕРА УЯЗВИМОСТЕЙ С ИСПОЛЬЗОВАНИЕМ БОЛЬШОЙ ЯЗЫКОВОЙ МОДЕЛИ

А.В. Матерухин, Д.Д. Сутягин, Ю.В. Бельшева, С.А. Калмыков

В статье предложено использовать большие языковые модели для автоматизации процесса генерации шаблонов проверок для сканера уязвимостей. Для проверки реализуемости этого подхода было проведено соответствующее экспериментальное исследование. В рамках этого исследования был проведен сбор и очистка данных из существующих шаблонов и соответствующих описаний уязвимостей, повышение качества наборов данных для обучения с использованием метода БЯМ-судьи, а также дообучение большой языковой модели с использованием оптимизаций. Полученная в результате дообучения модель продемонстрировала синтаксически и семантически правильную генерацию шаблонов для обнаружения уязвимостей. Полученные результаты могут быть использованы на практике для уменьшения временного разрыва между публикацией информации о новой уязвимости и её фактическим обнаружением в инфраструктуре компании.

Ключевые слова: информационная безопасность, автоматическое сканирование уязвимостей, технологии искусственного интеллекта.

Введение

Современная практика обеспечения информационной безопасности вычислительной инфраструктуры крупных компаний показывает, что одной из важнейших задач, на решение которых направлены усилия специалистов по информационной безопасности, является задача уменьшения временного разрыва между публикацией информации о новой уязвимости и её фактическим обнаружением в инфраструктуре компании.

Для минимизации продолжительности этого временного разрыва используются различные инструментальные средства для быстрой и автоматизированной проверки инфраструктуры компании на уже известные уязвимости. Важнейшим классом таких инструментальных средств являются автоматизированные сканеры уязвимостей. Часто используемым представителем этого класса является программное обеспечение с открытым кодом Nuclei [1, 2]. Это гибкий инструмент, который работает с различными протоколами (например, TCP, DNS, HTTP), использует шаблоны для проверок и обладает гибкостью благодаря настраиваемым шаблонам на языке YAML (далее: Nuclei-шаблоны или шаблоны). Важно отметить, что процесс создания

шаблонов требует активного участия в этом процессе специалиста по информационной безопасности – для каждой новой известной уязвимости требуется написать свой специфический шаблон, применимый в условиях конкретной инфраструктуры. Как отмечается в работе [3], полагаться на общие правила недостаточно: «распространённое решение – вручную создавать специализированные правила сканирования для конкретных проектов».

Авторы настоящей работы полагают, что для автоматизации процесса генерации шаблонов проверок для сканера уязвимостей возможно использовать большую языковую модель (далее: БЯМ или модель), модернизированную для решения этой задачи с помощью дообучения модели на специально сформированном наборе данных из корректных Nuclei шаблонов. Такой подход, по мнению авторов настоящей статьи, позволит получить модели для эффективной генерации корректных Nuclei шаблонов для новых уязвимостей.

Далее в статье изложены результаты экспериментального исследования предложенного авторами подхода. В качестве исходных данных использовались шаблоны и данные об уязвимостях из официального репозитория Nuclei, которые, в свою очередь, основаны на информации из базы данных CVE (Common Vulnerabilities

and Exposures) - общедоступной базы данных, содержащей информацию об известных уязвимостях в программном обеспечении. Для идентификации уязвимостей использовались уникальные идентификаторы из базы данных CVE - CVE ID.

1. Материалы и методы

Процесс начальной подготовки данных для дообучения модели изображен на рис. 1.

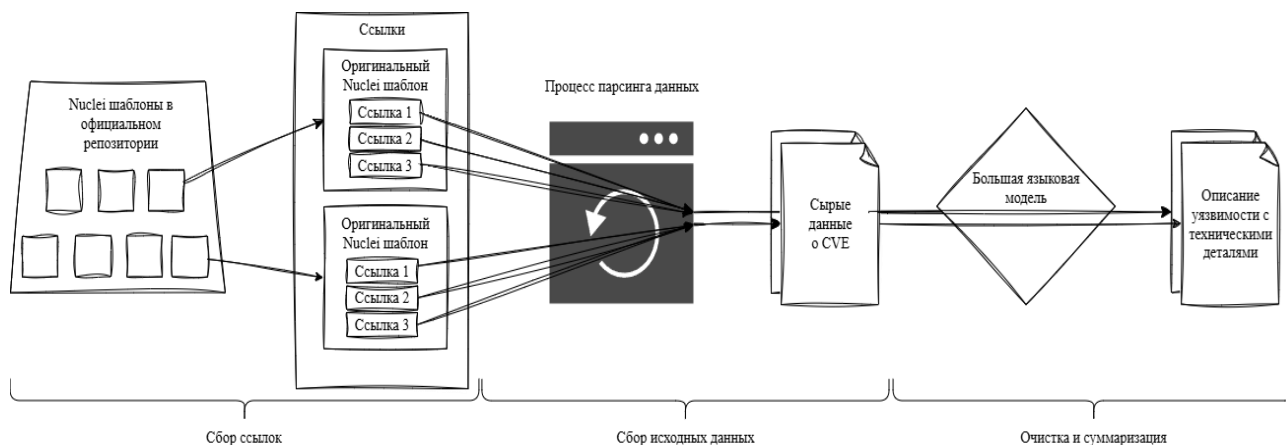


Рис. 1. Конвейер подготовки данных

- На первом этапе выполнялась выгрузка всех доступных шаблонов Nuclei из официального репозитория и извлечение из них метаданных из поля `info.references`, содержащего ссылки на описания уязвимостей.

- На втором этапе данных был выполнен парсинг данных со страниц с описаниями уязвимостей, извлеченных из поля `info.references`. Результат парсинга - необработанные файлы со всей информацией со страниц, как полезной, так и бесполезной.

- На заключительном третьем этапе этого процесса происходила очистка и суммаризация (под которой здесь понимается создание краткого содержания исходного текста с сохранением его основного смысла и информации) полученных данных с помощью модели. Следует отметить, что такая техника суммаризации данных уже была описана в работе [4]. Модели была поставлена задача описать подробную техническую

информацию по уязвимости, следуя заданной четкой структуре: обзор, описание, детали эксплуатации, масштаб влияния и методы обнаружения. В процессе суммаризации объединялись вся информация, образуя единый документ с «чистым» описанием уязвимости с полным её обзором по собранной информации.

Итог процесса начальной подготовки данных для дообучения модели – набор пар «суммаризированное текстовое описание - оригинальный Nuclei шаблон».

Далее авторами настоящей работы был организован процесс накопления качественного набора данных. Необходимость этого процесса была обоснована в [5]. Согласно [5] из множества пар (описание - шаблон) для дообучения следует отобрать самые качественные пары, так как именно они значительно улучшают результаты дообучения. Схема процесса накопления качественного набора данных изображена на рис. 2.

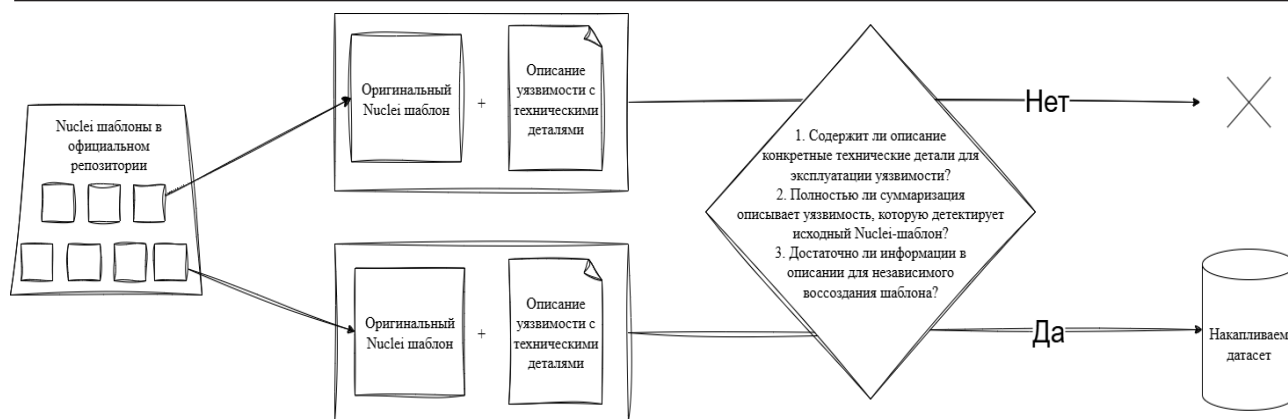


Рис. 2. Процесс накопления качественного набора данных

В настоящей работе для отбора таких пар была использована альтернатива ручной оценке сгенерированных текстов, описанная в работе [6], которая минимизирует влияние человеческого фактора – метод «БЯМ как судья» (далее: БЯМ-судья). БЯМ-судья оценивает полноту информации о технических деталях уязвимости, полностью ли сгенерированная суммаризация описывает ту уязвимость, которую детектирует исходный Nuclei шаблон, а также достаточно ли представленной информации для независимой генерации шаблона. Только пары, на которые БЯМ выдал положительные ответы по этим критериям, попадают в финальный набор данных.

Следует отметить, что из 9892 исходных пар после применения метода «БЯМ как судья» осталось 1433 пары, из которых 1333 использовались для обучения, а 100 – для теста.

Базовая идея предлагаемого авторами подхода – это дообучение модели. Необходимость дообучения вызвана тем, что для решения задачи генерации Nuclei шаблонов БЯМ общего назначения явно недостаточна. Необходимо научить модель понимать семантику описаний уязвимостей и генерировать код в соответствии с синтаксисом языка YAML. Следует отметить, что классическое полное дообучение всех параметров модели является крайне ресурсозатратным. Для уменьшения этой ресурсозатратности авторами настоящей статьи было принято решение использовать более эффективный

подход — параметро-эффективное дообучение, а именно метод LoRA (Low-Rank Adaptation), применение которого описано в [7]. Как показано в [7], LoRA позволяет достичь результатов, сопоставимых или даже превосходящих полное дообучение, при этом существенно снижая требования к вычислительным ресурсам и памяти. Суть метода LoRA заключается в том, что исходные веса предобученной модели остаются неизменными, а в слои трансформера внедряются дополнительные, низкоранговые обучаемые матрицы, что и позволяет на несколько порядков сократить число настраиваемых параметров, сохранив при этом качество генерации [7].

В качестве базовой модели была выбрана модель Qwen2.5-7B-base на 7 миллиардов параметров. Процесс дообучения был организован с помощью фреймворка LLaMA-Factory. Несмотря на то, что метод LoRA значительно сокращает число обучаемых параметров, дообучение модели размером 7 миллиардов параметров всё ещё требует значительных вычислительных ресурсов. На системном уровне для эффективного управления памятью и распределения нагрузки использовался фреймворк DeepSpeed с конфигурацией ZeRO Stage 3. На операционном уровне, для ускорения самих вычислений, был интегрирован Liger Kernel — набор высокопроизводительных Triton-ядер, оптимизирующих ключевые математические операции в трансформерах. Согласно [8], Liger Kernel обеспечивает

прирост пропускной способности обучения стандартными реализациями. Полный перечень гиперпараметров, использованных для дообучения, представлен в табл. 1.

Таблица 1

Ключевые гиперпараметры обучения модели		
Параметр	Описание	Значение
LoRA rank	Ранг низкоранговых матриц адаптации	512
LoRA α (alpha)	Масштабирующий коэффициент адаптации	1024
LoRA dropout	Вероятность дропаута в LoRA слоях	0
Target modules	Модули, к которым применена LoRA-адаптация	All
fp16	Использование полуплавающей точности для ускорения обучения	True
Gradient accumulation steps	Количество шагов аккумуляции градиентов	4
Learning rate	Начальная скорость обучения	5.0×10^{-6}
Scheduler	Тип планировщика скорости обучения	Cosine
Epochs	Количество эпох обучения	3
Warmup steps	Количество шагов «разогрева»	100
Max sequence length	Максимальная длина входной последовательности	8192

2. Результаты

На рис. 3 показан график изменения значения функции потерь в процессе обучения. На начальном этапе (приблизительно до 75-го шага) происходит резкое снижение потерь, за которым следует замедление и выход на плато, где итоговое значение функции потерь стабилизируется в районе значения 0.8. Для искусственных нейросетей с топологией «трансформер» характерно прохождение через фазу «плато потерь», которая часто предшествует резкому улучшению производительности. В данном случае зафиксирован типичный вход в фазу «плато» как раз перед резким улучшением производительности. Наблюдаемое поведение согласуется с наблюдениями в исследовании [9].

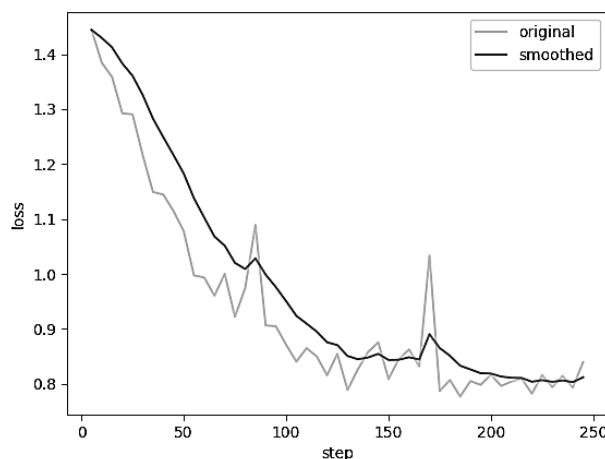


Рис. 3. Динамика функции потерь в процессе обучения

Таким образом, по мнению авторов настоящей статьи, динамика функции потерь в процессе обучения свидетельствует об успешной сходимости модели. Выход на плато, по мнению авторов настоящей статьи, указывает не на остановку обучения, а на завершение этапа усвоения простых

паттернов и переход к более сложной и медленной фазе оптимизации.

Для оценки результатов дообучения модели был проведён ручной анализ 100 сгенерированных дообученной моделью примеров. Была выявлена работоспособность и корректность сгенерированных шаблонов. Модель успешно генерировала синтаксически и

семантически корректные шаблоны на основе описаний уязвимостей. Например, для уязвимости CVE-2022-29078, шаблон которой не участвовал при формировании датасета и в процессе дообучения модели, был сгенерирован YAML-шаблон (рис. 4), практически идентичный оригинальному (рис. 5).

```
id: CVE-2022-29078

info:
  name: EJS Server-Side Template Injection RCE
  author: 0x00-0x00
  severity: critical
  description: |
    The vulnerability arises in the EJS (Embedded JavaScript Templates) package 3.1.6 for Node.js, allowing server-side template injection through the `settings[view options][outputFunctionName]` parameter. This parameter is parsed as an internal option, allowing an attacker to overwrite the `outputFunctionName` option with arbitrary code, which gets executed during template compilation.
  reference:
    - https://nvd.nist.gov/vuln/detail/CVE-2022-29078
    - https://eslam.io/posts/ejs-server-side-template-injection-rce/
    - https://github.com/mde/ejs/releases/tag/v3.1.7
    - https://security.netapp.com/advisory/ntap-20220804-0001/
  classification:
    cvss-metrics: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
    cvss-score: 9.8
    cve-id: CVE-2022-29078
    cwe-id: CWE-78
    epss-score: 0.00010
    epss-percentile: 0.00010
    cpe: cpe:2.3:a:mde:ejs:3.1.6:*:*:*:*:*:*
  metadata:
    max-request: 1
    vendor: mde
    product: ejs
    shodan-query: http.title:"Express"
  tags: cve,cve2022,ejs,express,server-side-template-injection,rce,netapp

http:
  - raw:
    - |
      GET /page?id=2&settings[view options][outputFunctionName]=x;process.mainModule.require('child_process').execSync('touch /tmp/pwned');s HTTP/1.1
      Host: {{Hostname}}
      User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/111.0
      Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
      Accept-Language: en-US,en;q=0.5
      Accept-Encoding: gzip, deflate
      Connection: close
      Upgrade-Insecure-Requests: 1

  matchers-condition: and
  matchers:
    - type: word
      part: body
      words:
        - "pwned"
      condition: and

    - type: status
      status:
        - 200
```

Рис. 4. Сгенерированный шаблон

id: CVE-2022-29078

info:

name: Node.js Embedded JavaScript 3.1.6 - Template Injection

author: For3stCo1d

severity: critical

description: |

Node.js Embedded JavaScript 3.1.6 is susceptible to server-side template injection via settings[view options][outputFunctionName], which is parsed as an internal option and overwrites the outputFunctionName option with an arbitrary OS command, which is then executed upon template compilation.

impact: |

Remote code execution can lead to unauthorized access, data leakage, and complete system compromise.

remediation: |

Upgrade to a patched version of Node.js Embedded JavaScript (3.1.7 or higher) to mitigate the vulnerability.

reference:

- <https://eslam.io/posts/ejs-server-side-template-injection-rce/>
- <https://github.com/miko550/CVE-2022-29078>
- <https://github.com/mde/ejs/commit/15ee698583c98dad456639d6245580d17a24baf>
- <https://nvd.nist.gov/vuln/detail/CVE-2022-29078>
- <https://github.com/mde/ejs/releases>

classification:

cvss-metrics: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

cvss-score: 9.8

cve-id: CVE-2022-29078

cwe-id: CWE-94

epss-score: 0.28707

epss-percentile: 0.96859

cpe: cpe:2.3:a:ejs:3.1.6:*:*:*:*:*:node.js:*:*

metadata:

max-request: 1

vendor: ej

product: ej

framework: node.js

tags: cve,cve2022,ssti,rce,ejs,nodejs,oast,intrusive,node.js

http:

- raw:

```

- |
  GET /page?id={{randstr}}&settings[view%20options][outputFunctionName]=x;process.mainModule.require(%27child_process%27).execSync(%27wget+http://{{interactsh-url}}%27);s HTTP/1.1
  Host: {{Hostname}}

```

matchers-condition: and

matchers:

- type: word

part: interactsh_protocol # Confirms the HTTP Interaction

words:

- http

- type: word

part: body

words:

- You are viewing page number

Рис. 5. Оригинальный шаблон

Все сгенерированные моделью шаблоны были оценены как корректные и соответствующие входному описанию. Однако следует отметить, что иногда модель дублировала строки без необходимости. Например, на рис. 6 видно, что в разделе path один и тот же путь повторён несколько раз подряд, что не имеет смысла, но не является ошибкой.

requests:

- method: GET

path:

- "{{BaseURL}}/test"
- "{{BaseURL}}/test"
- "{{BaseURL}}/test"

Рис. 6. Пример дублирования строки

Заключение

Авторами настоящей статьи предложен и проверен экспериментально подход, позволяющий автоматизировать создание шаблонов для сканера уязвимостей с помощью дообученной БЯМ. Описанная реализация предложенного подхода включает в себя сбор и очистку данных из существующих шаблонов и соответствующих описаний уязвимостей, фильтрацию наборов с использованием метода БЯМ-судьи, и собственно дообучение с использованием оптимизаций. Полученная модель демонстрирует синтаксически и семантически правильную генерацию шаблонов для обнаружения уязвимостей.

Полученные результаты показывают, что БЯМ при целенаправленном дообучении на качественном узкоспециализированном наборе данных может эффективно решать задачу генерации шаблонов проверок для сканера уязвимостей.

Список литературы

1. Dung N.Q. Development of Nuclei Templates for Security Vulnerabilities Detection in WordPress / N.Q. Dung // Journal of Electrical Systems. 2024. Vol. 20, No. 11s. P. 2324–2335;
2. Nuclei // GitHub: современный высокопроизводительный сканер уязвимостей, использующий простые шаблоны на основе YAML. 2020. URL: <https://github.com/projectdiscovery/nuclei> (дата обращения: 13.07.2025);
3. Hu J., Jin X., Zeng Y., [et al.]. QLPro: Automated Code Vulnerability Discovery via LLM and Static Code Analysis Integration // arXiv: бесплатный сервис распространения и архив с открытым доступом. 2025. URL: <https://arxiv.org/abs/2506.23644> (дата обращения: 08.07.2025);
4. From CVE to Template: The Future of Automating Nuclei Templates with AI // ProjectDiscovery, Inc. 2024. URL: <https://projectdiscovery.io/blog/future-of-automating-nuclei-templates-with-ai> (дата обращения: 10.07.2025);
5. Pang J., Wei J., Shah A.P., [et al.]. Improving Data Efficiency via Curating LLM-Driven Rating Systems // arXiv: бесплатный сервис распространения и архив с открытым доступом. 2024. URL: <https://arxiv.org/abs/2410.10877> (дата обращения: 08.07.2025);
6. LLM-as-a-judge: a complete guide to using LLMs for evaluations // Evidently AI. URL: <https://www.evidentlyai.com/БЯМ-guide/БЯМ-as-a-judge> (дата обращения: 23.07.2025);
7. Hu E.J., Shen Y., Wallis P., [et al.]. LoRA: Low-Rank Adaptation of Large Language Models // arXiv: бесплатный сервис распространения и архив с открытым доступом. 2021. URL: <https://arxiv.org/abs/2106.09685> (дата обращения: 09.07.2025);
8. Hsu P.-L., Dai Y., Kotkapalli V., [et al.]. Liger Kernel: Efficient Triton Kernels for LLM Training // arXiv: бесплатный сервис распространения и архив с открытым доступом. 2024. URL: <https://arxiv.org/abs/2410.10989> (дата обращения: 18.07.2025);
9. Gopalani P., Hu W. What Happens During the Loss Plateau? Understanding Abrupt Learning in Transformers // arXiv: бесплатный сервис распространения и архив с открытым доступом. 2025. URL: <https://arxiv.org/abs/2506.13688> (дата обращения: 17.07.2025).

Московский государственный университет геодезии и картографии
Moscow State University of Geodesy and Cartography

Поступила в редакцию 17.10.2025

Информация об авторах

Матерухин Андрей Викторович – д-р техн. наук, декан факультета геоинформатики и информационной безопасности, Московский государственный университет геодезии и картографии (105064, Россия, г. Москва, Гороховский переулок, 4), e-mail: materukhinav@miigaik.ru, ORCID: <https://orcid.org/0000-0002-9576-9925>.

Сутягин Даниил Денисович – старший преподаватель кафедры информационно-измерительных систем, Московский государственный университет геодезии и картографии (105064, Россия, г. Москва, Гороховский переулок, 4), e-mail: steeji.rat@gmail.com.

Бельшева Юлия Владимировна – старший преподаватель кафедры информационно-измерительных систем, Московский государственный университет геодезии и картографии (105064, Россия, г. Москва, Гороховский переулок, 4), e-mail: belysheva@miigaik.ru.

Калмыков Святослав Алексеевич – преподаватель кафедры информационно-измерительных систем, Московский государственный университет геодезии и картографии (105064, Россия, г. Москва, Гороховский переулок, 4), e-mail: s_kalmykov@miigaik.ru, ORCID: <https://orcid.org/0009-0004-7807-2415>.

AUTOMATION OF NUCLEI TEMPLATE GENERATION USING A FINE-TUNED LARGE LANGUAGE MODEL

A.V. Materukhin, D.D. Sutyagin, Yu.V. Belysheva, S.A. Kalmykov

The article addresses the issue of delayed response to cyber threats caused by the manual process of creating templates for the Nuclei vulnerability scanner. To solve this problem, a method for automating template generation using a parameter-efficient fine-tuning Large Language Model (LLM) is proposed. The research process consisted of three main stages. First, a data preparation pipeline was developed, in which information about vulnerabilities was extracted from the official Nuclei repository and summarized using an LLM. On the next step to ensure the high quality of the training dataset, the «LLM-as-a-Judge» approach was applied, where another model selected the best «template–description» pairs based on three criteria: completeness of technical information, consistency between the description and the template, and sufficiency of data for independent template generation. As a result, 1,433 templates were selected from 9,892 originals. At the final stage, the Qwen2.5-7B-base model was fine-tuned using the parameter-efficient LoRA method along with DeepSpeed and Liger Kernel optimizations. The results demonstrated successful model convergence, and qualitative evaluation on the test dataset confirmed that the model generates syntactically and semantically correct, functional Nuclei templates that are nearly identical to the original ones.

Keywords: Information security, automation, vulnerability scanning, artificial intelligence technologies.

Submitted 17.10.2025

Information about the authors

Andrey V. Materukhin – Doctor of Technical Sciences, Dean of the Faculty of Geoinformatics and Information Security, Moscow State University of Geodesy and Cartography, 4 Gorokhovskiy Lane, Moscow, 105064, Russia. E-mail: materukhinav@miigaik.ru, ORCID: <https://orcid.org/0000-0002-9576-9925>.

Daniil D. Sutyagin – Senior Lecturer, Department of Information and Measuring Systems, Moscow State University of Geodesy and Cartography, 4 Gorokhovskiy Lane, Moscow, 105064, Russia. E-mail: steeji.rat@gmail.com.

Yulia V. Belysheva – Senior Lecturer, Department of Information and Measuring Systems, Moscow State University of Geodesy and Cartography, 4 Gorokhovskiy Lane, Moscow, 105064, Russia. E-mail: belysheva@miigaik.ru.

Svyatoslav A. Kalmykov – Lecturer, Department of Information and Measuring Systems, Moscow State University of Geodesy and Cartography, 4 Gorokhovskiy Lane, Moscow, 105064, Russia. E-mail: s_kalmykov@miigaik.ru, ORCID: <https://orcid.org/0009-0004-7807-2415>.